

Napredni modeli i baze podataka

Predavanja
Listopad 2025.

**Pretraživanje teksta u relacijskim sustavima za
upravljanje bazama podataka**

Motivacijski primjer 1

peer assessment in university te x +

scholar.google.hr/scholar?hl=hr&as_sdt=0%2C5&q=peer+assessment+in+university+teaching&btnG=

Google Znalac

peer assessment in university teaching

Članci

Oko 4.410.000 rezultata (0,16 s)

Bilo kad

Od 2024

Od 2023

Od 2020

Odabrani raspon...

Razvrstaj po važnosti

Razvrstaj po datumu

Bilo koja

Članci s recenzijom

☐ uključi patente

☒ uključi citate

☒ Stvori obavijest

Peer assessment in university teaching: evaluating seven course designs
I Van den Berg, W Admiraal, A Pilot - *Assessment & Evaluation in ...*, 2006 - Taylor & Francis
... positive effects of **peer assessment**, especially on writing in higher education. In this study, a definition and typology of **peer assessment** is being proposed. **Peer assessment** is defined ...
☆ Spremi Citiraj Spominje se 294 puta Srodni članci Svih 8 inačica [PDF] tandfonline.com Full View

Peer assessment between students in colleges and **universities**
K Topping - *Review of educational Research*, 1998 - journals.sagepub.com
... **peer assessment** is of adequate reliability and validity in a wide variety of applications. **Peer assessment** of writing and **peer assessment** ... the effects of **teacher assessment**. Evidence for ...
☆ Spremi Citiraj Spominje se 3442 puta Srodni članci Svih 12 inačica Web of Science: 1095 [HTML] proquest.com

Peer assessment
KJ Topping - *Theory into practice*, 2009 - Taylor & Francis
... in their written **peer assessments** to the **teacher**—six from each group of three people. The **teacher** compares the two **peer assessments** for each essay. Where the two **assessments** are ...
☆ Spremi Citiraj Spominje se 1491 puta Srodni članci Svih 6 inačica Web of Science: 391 [PDF] tandfonline.com Full View

Design principles and outcomes of **peer assessment** in higher education
I Van den Berg, W Admiraal, A Pilot - *Studies in Higher education*, 2006 - Taylor & Francis
... **peer assessment** of written assignments in the context of **teaching** history at **university** level. To discover features yielding optimal results, several **peer assessment** designs were ...
☆ Spremi Citiraj Spominje se 301 puta Srodni članci Svih 8 inačica Web of Science: 90 [PDF] tandfonline.com Full View

Motivacijski primjer 2

STOP PREPISIVAČIMA NA SVEUČILIŠTU

Softverom Turnitin u lov na plagijatore

Glas Slavonije

Antiplagijatorski softver koriste sve sastavnice osječkog sveučilišta (Foto: DAVOR KIBEL)

- Računalni program Turnitin u bazama radova i na više od 61 bilijuna web stranica tražit će slične rečenice, reference i citate
- Zakonska obaveza je javno objavljivati diplomske i doktorske radove, a Sveučilišta planiraju kontrolu plagiranja provesti uz pomoć Turnitina (<http://turnitin.com/>).

Motivacijski primjer 2

Assignment Inbox

turnitin.com/assignment/type/paper/inbox/153255867?lang=en_us

Liljana Brkić | User Info | Messages | Instructor | English | Community | Help | Logout

turnitin

Edit Assignment | GradeMark Report | Students | Libraries | Discussion

NOW VIEWING: HOME > SIMILARITY REPORT > ZAVRŠNI RAD

About this page

This is your assignment inbox. To view a paper, select the paper's title. To view a Similarity Report, select the paper's Similarity Report icon in the similarity column. A ghosted icon indicates that the Similarity Report has not yet been generated.

Submit

Search

All Papers

Download All

Move to

☐	Author	Title	Submission ID	Uploaded	Viewed	Similarity	Flags	Options
☐	Tomislav Grudić	Zavrsni_rad_Tomislav_Gr...	2449256954	September 9, 2024		1%	--	...
☐	Nikola Banković	Banković_Nikola_završni...	2401874516	August 24, 2024		5%	--	...
☐	Karlo Vrančić	kv54199-zavrsni.docx	2413531207	July 7, 2024		2%	--	...
☐	Tea Četojević-Tisaj	proba.docx	2400603019	June 14, 2024		--	--	...

Chart 1: Levels of Unoriginal Content, All U.S. Higher Education

Year	Level of Unoriginal Content
Year 1	0
Year 2	+1.8%
Year 3	-13.2%
Year 4	-25.5%
Year 5	-39.1%

izvor

© ZPR-FEP

4

Zbog čega je pretraživanje teksta izazov?

- Tekst pisan prirodnim jezikom podliježe pravopisnim i gramatičkim pravilima jezika. Mnogo jezika, mnogo pravila.
- Procjenjuje se da 80% poslovnih podataka pripada kategoriji nestrukturiranih. Većina je u tekstualnom obliku. Upravljanje (pohrana, pretraga) velikim količinama podataka predstavlja tehnički i tehnologijski izazov.
- Istraživanje u području pretraživanja teksta zahtijeva znanja eksperta iz područja jezikoslovlja (lingvistike) i računarstva.
- Tekst pisan prirodnim jezikom je često **nejasan** i **dvosmislen**

Nejasan, dvosmislen

- Nejasno je što je korisniku jeftino

Rezultati pretrage za **jeftin skuter**

1 2 3 4 5 6 7-8 SLJEDEĆA »

148 oglasa Sortiraj Relevantnosti ▼

Njuškalo oglasi



JEFTINI BIKIKLI I SKUTERI !! ★

Prodajem 3 bicikla i 2 skutera. Bicikli i skuteri su stari cca. 1-2 godine sve radi ispravno.
Objavljen: 14.08.2015.

300 kn



PRODAJE SE SKUTER 200 KUBA PIAGGIO hitno i jeftino!! ★

Rabljeni motor, 35000 km
Godina proizvodnje: 2005.
Objavljen: 04.08.2015.

1.750 € ~ 13.239 kn

- Dvosmislen
 - Na prodaju je kuća mog prijatelja u Splitu.
 - Kuća mog prijatelja u Splitu je na prodaju.
- Je li prijatelj u Splitu ili kuća?

Pretraživanje teksta (Full Text Search - FTS)

- Traženje dokumenata koji zadovoljavaju postavljeni **uvjet** i njihovo rangiranje u skladu sa **sličnošću** dokumenta s postavljenim uvjetom.
- **Uvjet** je obično niz riječi, a **sličnost** je brojčana vrijednost koja je u najjednostavnijoj implementaciji povezana s frekvencijom riječi iz uvjeta u pretraživanom dokumentu.

Zašto pretraživati tekst u relacijskim bazama podataka?

- Zbog velike količine tekstualnih podataka pohranjenih u relacijskim bazama podataka
- Razlozi? Povijesni. Naslijeđe.
- Svi vodeći proizvođači RSUBP imaju implementiranu podršku za pretragu teksta.

Tekst u relacijskim bazama podataka i standardni SQL

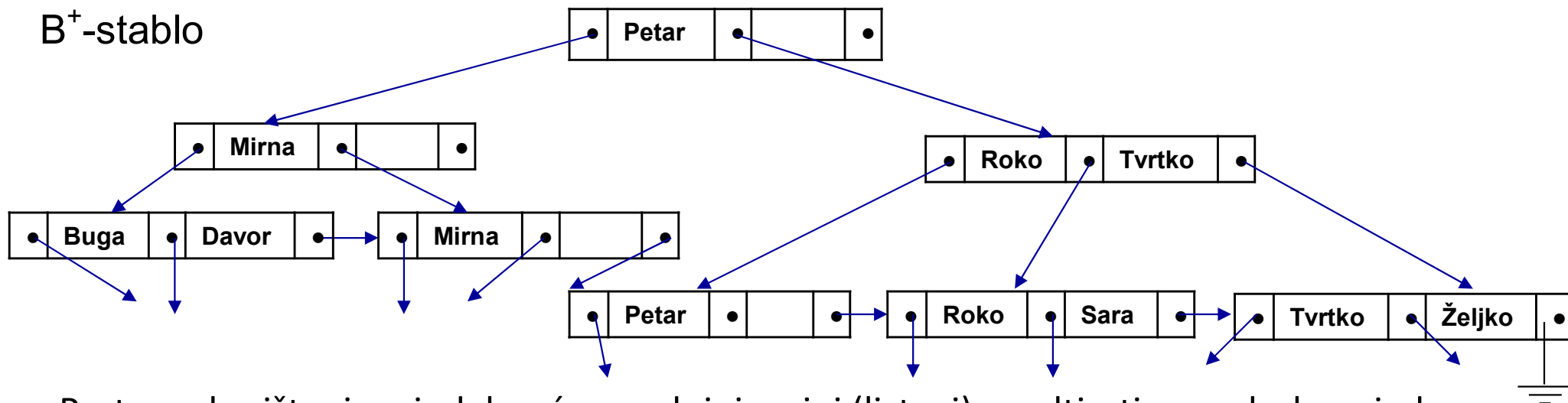
- tipovi podatka
 - CHARACTER, CHARACTER VARYING, CHARACTER LARGE OBJECT; NATIONAL CHARACTER, NATIONAL CHARACTER VARYING, NATIONAL CHARACTER LARGE OBJECT
 - npr. PostgreSQL: CHAR(*n*), VARCHAR(*n*), TEXT
IBM Informix: CHAR (*m,r*), NCHAR(*m,r*), VARCHAR (*m,r*),
LVARCHAR (*m,r*), TEXT
- funkcije:
 - SUBSTRING, UPPER, LOWER, TRIM, CHAR_LENGTH, OCTET_LENGTH
- operatori:
 - ||,
LIKE, NOT LIKE (specijalni znakovi '_' i '%')
SIMILAR, NOT SIMILAR

Nedostaci standardnog relacijskog SUBP u pretraživanju teksta

- Ne postoje efikasni indeksi pa pri procesiranju svakog upita treba „obraditi” svaki dokument $\Longrightarrow$ sporo

```
CREATE INDEX imeOsobaIdx on osoba (ime);
```

B⁺-stablo



- Pretraga korištenjem indeksa će u zadnjoj razini (listovi) rezultirati pronalaskom jedne (početne) vrijednost ključa;
ovisno o vrsti uvjeta i vrsti indeksa (unique ili ne) broj pronađenih ntorki može biti veći od 1

```
SELECT *  
  FROM osoba  
 WHERE ime = 'Ana';
```

```
SELECT *  
  FROM osoba  
 WHERE ime LIKE '%ana';
```

```
SELECT *  
  FROM osoba  
 WHERE ime LIKE 'Ana%';
```

```
SELECT *  
  FROM osoba  
 WHERE ime LIKE '%ana%';
```

koristi index

ne koristi
index

Nedostaci standardnog relacijskog SUBP u pretraživanju teksta

- Ne postoji lingvistička podrška
 - nema načina da se prepoznaju različite inačice iste riječi, npr. *podatak* i *podatkovni*
 - nemoguće je zanemariti veznike, prijedloge i ostale riječi koje se često pojavljuju u tekstovima a imaju malo ili nikakvo semantičko značenje
 - nemoguće je dovesti u vezu riječi istog značenja (sinonimi)
- Ne postoji rangiranje rezultata što rezultat s mnogo dokumenata čini neupotrebljivim

Pristupi pri pretraživanju teksta

1. Točno podudaranje
 - *uvjet* i tekst se u cijelosti podudaraju, identični su
 - *uvjet* je sadržan u dijelu teksta (djelomično podudaranje)
2. Podudaranje temeljem morfologije, sintakse i semantike jezika
 - koriste se tzv. gramatički algoritmi koji pronalaze podudarnost između *uvjeta* i teksta temeljem normaliziranog oblika riječi (korijena, leksema), temeljem sinonima i td.
Npr. riječi *podatak*, *podatci*, *podatka*, *podatkom*, *podatku*,... imaju jednak normalizirani oblik
3. Približno podudaranje (fuzzy)
 - Algoritmi temeljeni na „udaljenosti” između znakovnih nizova
 - Q-Gram algoritmi
 - Fonetsko podudaranje

PostgreSQL: Pretraživanje teksta

1. Točno podudaranje

- =
- LIKE, NOT LIKE
- SIMILAR TO (SQL:1999 (SQL3))
- Regularni izrazi (*regular expressions* – *regex*)

Standardne mogućnosti SQL-a
u relacijskim SUBP

2. Podudaranje temeljem morfologije, sintakse i semantike jezika

- TSVector, TSQuery tipovi podataka, @@ operator

3. Približno podudaranje

- Algoritmi temeljeni na „udaljenosti” između znakovnih nizova
 - Levenshtein funkcija
- Q-Gram algoritmi
 - % operator, similarity funkcija
- Fonetsko podudaranje
 - Soundex, Metaphone funkcije

Pretraživanje teksta temeljem morfologije, sintakse i semantike jezika

- **Morfologija** ili **oblikoslovlje** je dio jezikoslovlja koji proučava načine na koji se riječi u nekom jeziku oblikuju i mijenjaju. Bavi se primjerice:
 - oblicima riječi i njihovim promjenama (deklinacija, konjugacija),
 - tvorbom riječi (prefiksi, sufixi, slaganje),
 - gramatičkim kategorijama (broj, rod, padež, vrijeme, ...),
 - razlikom između osnovnih (leksičkih) i izvedenih oblika riječi.
- **Sintaksa** je dio jezikoslovlja koji proučava:
 - redoslijed riječi u rečenici,
 - odnos između subjekta, predikata i objekta,
 - vrste rečenica i njihovu strukturu,
 - razlike u strukturi rečenica među jezicima.
- **Semantika** je dio jezikoslovlja koji proučava značenja:
 - što znače riječi, fraze i rečenice,
 - kako kontekst utječe na značenje,
 - na koji se način prenosi eksplicitno i implicitno značenje.
 - ...

Pretraživanje teksta temeljem morfologije, sintakse i semantike jezika

Morfem	najmanja jezična jedinica koja ima i svoj oblik i svoje značenje primjer: u gleda-m, jedinica m ima značenja: „prvo lice“, „jednina“, „sadašnje vrijeme“.
Korijenski morphem	obavezni, neizostavni dio oblika riječi, nositelj temeljnog značenja (npr. kuć u kući, majk u majci; složenice imaju više korijena npr. <u>vodopad</u>)
Afiks	svaki morphem koji nije korijenski, koji se pričvršćuje na "bazu" • prefiksi (pra-djed, na-učiti, pre-po-znati) • sufiksi (npr. vod-a, vod-e; radi-m, radi-š) • interfiksi (glav-o-bolja, nog-o-met, kaž-i-prst, cjepi-i-dlaka) • infiksi, cirkumfiksi, transfiksi, superfiksi

Zbog čega nam je morfologija, sintaksa i semantika jezika važna?

- Kada **uvjet** npr. sadrži riječ **sreća** očekujemo da su u rezultatu dokumenti koji sadrže riječ **sreća** ali i riječi
 - **sretnik, sretnica, nesretnik,...** $\Rightarrow$ morfologija
 - **radost, veselje, blaženstvo,...** $\Rightarrow$ semantika (sinonimi,...)
- Da bi to bilo moguće moramo poznavati morfološke operacije (načine građenja oblika riječi neovisno o svrsi te gradnje), te sintaktička i semantička pravila jezika
- Jezikoslovlje nije tema ovog predmeta.
- Navedeni pojmovi i primjeri samo ilustriraju problem traženja i prepoznavanja **uvjet-a** u pretraživanom tekstu.

Pretraživanje teksta temeljem morfologije, sintakse i semantike jezika

- Tekst/dokument je (multi) skup riječi (bag of words)
- Efikasna pretraga teksta podrazumijeva prethodnu obradu teksta (dokumenta):
 - pretprocesiranje teksta i rastavljanje na sastavne dijelove (riječi, brojevi, tagovi, „bjeline“, url,...)
 - uklanjanje „riječi“ koje ne doprinose razlikovanju teksta od drugih
 - predstavljanje različitih morfoloških oblika „iste“ riječi jednim jedinstvenim oblikom
(npr. *stan* u riječima *stanar*, *stanari*, *stanarski*,...
ili *bank* za riječi *banking*, *banked*, *banks*, *banks'* i *bank's*)
 - prepoznavanje različitih riječi jednakog značenja
 - ...
 - pohranu obrađenog teksta u obliku prikladnom za pretragu

PostgreSQL: FTS (temeljem morfologije, sintakse i semantike jezika)

Podržan je pomoću:

1. Novih tipova podataka TSVector i TSQuery te skupa pravila za transformaciju teksta/dokumenata i upita u njihove FTS reprezentante
 - parser, rječnici
 - funkcije za transformaciju tekstualnih podataka u TSVector i TSQuery (*to_tsVector*, *to_tsQuery*, *plainto_tsQuery*)
 2. FTS operatora za rad s TSVector i TSQuery
 - @@
 3. Funkcija za rangiranje rezultata
 - ts_rank, ts_rank_cd
- TSVector
 - podatkovni tip za reprezentaciju dokumenta
 - TSQuery
 - podatkovni tip za reprezentaciju tekstualnog uvjeta pretage

PostgreSQL: Parser

- Razdvaja izvorni tekst na tokene i utvrđuje tip tokena
- Tipovi tokena koje parser prepoznaje unaprijed su definirani
- Ugrađeni parser PostgreSQL-a razlikuje 23 vrste tokena: asciiword, word, numword, email, protocol, url, host, file, tag, blank,...
- Ne modificira izvorni tekst
- *ts_debug* funkcija za testiranje parametara pretrage; za svaki token predanog teksta, prikazuje informaciju u skladu s trenutnim parametrima pretrage

```
SELECT alias, description, token
FROM ts_debug ('english',
               'The Dancing Ladies.');
```

alias	description	token
asciiword	Word, all ASCII	The
blank	Space symbols	
asciiword	Word, all ASCII	Dancing
blank	Space symbols	
asciiword	Word, all ASCII	Ladies
blank	Space symbols	.

```
SELECT alias, description, token
FROM ts_debug ('english',
               'http://www.fer.unizg.hr/o_fakultetu')
```

alias	description	token
protocol	Protocol head	http://
url	URL	www.fer.unizg.hr/o_fakultetu
host	Host	www.fer.unizg.hr
url_path	URL path	/o_fakultetu

PostgreSQL: Rječnici

- Služe za
 - uklanjanje „riječi” koje nemaju semantičko značenje u tekstu
 - predstavljanje različitih morfoloških oblika „iste” riječi jednim jedinstvenim oblikom (lexem) (svođenje riječi na tzv normalizirani oblik koji se ne mora nužno podudarati s korijenskim oblikom)
 - prepoznavanje različitih riječi jednakog značenja (sinonimi)
 - ...
- Time se reducira veličina reprezentacije teksta i postižu dobra svojstva pretraživanja teksta (TSVector je sažeta verzija izvornog teksta)
- PostgreSQL ima nekoliko tipova rječnika:
 - Simple Dictionary
Velika slova svodi na mala. Ne uklanja stop riječi.
 - Synonym Dictionary
Različite riječi jednakog značenja zamjenjuje reprezentantnom rječju.
 - Thesaurus Dictionary
Prepoznaje fraze.
 - iSpell Dictionary i Snowball Dictionary
Svodi riječi na korijenski oblik (stemming) i uklanjaju stop riječi

PostgreSQL: Rječnici

- Rječnik je program koji za ulazni *token* vraća:
 - polje normaliziranih oblika (leksema) ako rječnik prepoznaje token (jedan token može imati više korijena)
 - prazno polje ako rječnik prepoznaje token, a token predstavlja stop riječ
 - NULL ako rječnik ne prepoznaje token

```
SELECT *  
FROM ts_debug('english', 'The Dancing Ladies');
```

alias	description	token	dictionaries	dictionary	lexemes
asciiword	Word, all ASCII	The	{english_stem}	english_stem	{}
blank	Space symbols		{}	[null]	[null]
asciiword	Word, all ASCII	Dancing	{english_stem}	english_stem	{danc}
blank	Space symbols		{}	[null]	[null]
asciiword	Word, all ASCII	Ladies	{english_stem}	english_stem	{ladi}

english_stem is a Snowball dictionary koji prepoznaje sve. Snowball dictionary template se zasniva na projektu Martina Portera, autora popularnog Porterovog algoritma za pronalaženje korijena riječi u engleskom jeziku.

Za testiranje ponašanja konkretnog rječnika može se koristiti ts_lexize funkcija

ts_lexize(dict regdictionary, token text) returns text[]

```
SELECT ts_lexize ('english_stem', 'Dancing');
```

ts_lexize
{danc}

```
SELECT ts_lexize ('simple', 'Dancing');
```

ts_lexize
{dancing}

PostgreSQL: Rječnici

- Različiti tokeni mogu rezultirati jednakom listom korijena

```
SELECT *  
FROM ts_debug ('english', 'happy happiness happily');
```

alias	description	token	dictionaries	dictionary	lexemes
asciiword	Word, all ASCII	happy	{english_stem}	english_stem	{happi}
blank	Space symbols		{}	[null]	[null]
asciiword	Word, all ASCII	happiness	{english_stem}	english_stem	{happi}
blank	Space symbols		{}	[null]	[null]
asciiword	Word, all ASCII	happily	{english_stem}	english_stem	{happili}

PostgreSQL: Rječnici

- PostgreSQL ima inicijalne rječnike za raširenije svjetske jezike.
- Postoje i predlošci koji se mogu koristiti za kreiranje novih rječnika.
- Novokreirani rječnici se konfiguriraju pomoću parametara.

```
CREATE TEXT SEARCH DICTIONARY name  
( TEMPLATE = template  
[, option = value [, ... ]] )
```

```
ALTER TEXT SEARCH DICTIONARY name  
( option [ = value ] [, ... ] )
```

```
CREATE TEXT SEARCH DICTIONARY croatian  
( template = snowball  
, language = croatian  
, stopwords = croatian);
```

General

Name: danish_stem

OID: 12686

Owner: postgres

Comment: snowball stemmer for danish language

Definition

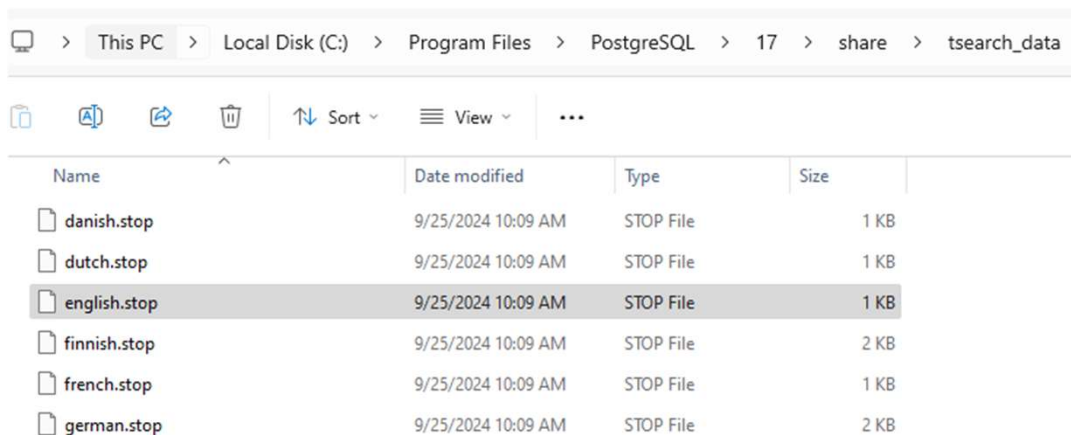
Template: snowball



ERROR: no Snowball stemmer available for language "croatian" and encoding "UTF8"

PostgreSQL: Stop (zaustavne) riječi

- Riječi koje se vrlo često pojavljuju, gotovo u svakom tekstu
- Ignoriraju se pri pretrazi teksta izuzev pri traženju fraza
- Npr. tekstovi na engleskom jeziku redovito sadrže članove "a" i "the" te bi njihovim uključivanjem u proces pretrage gotovo svi dokumenti zadovoljili uvjet pretrage
- Utječu na rangiranje dokumenata – vidi slajd o rangiranju kasnije
- Liste stop riječi za raširenije svjetske jezike dolaze s inicijalnom instalacijom PostgreSQL-a



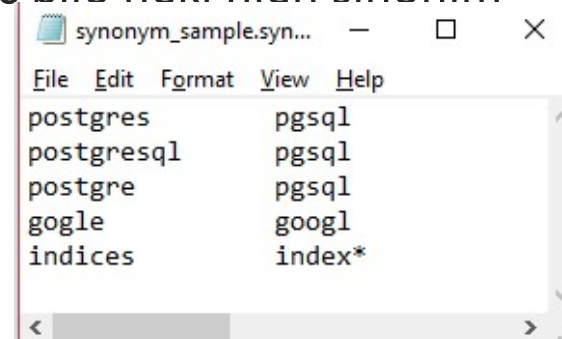
Name	Date modified	Type	Size
danish.stop	9/25/2024 10:09 AM	STOP File	1 KB
dutch.stop	9/25/2024 10:09 AM	STOP File	1 KB
english.stop	9/25/2024 10:09 AM	STOP File	1 KB
finnish.stop	9/25/2024 10:09 AM	STOP File	2 KB
french.stop	9/25/2024 10:09 AM	STOP File	1 KB
german.stop	9/25/2024 10:09 AM	STOP File	2 KB

```
i  
me  
my  
myself  
we  
our  
ours  
ourselves  
you  
your  
yours  
yourself  
yourselves  
he  
him  
his  
himself  
she  
her  
hers  
herself  
it  
its  
"english.stop"
```

PostgreSQL: Rječnik sinonima

- Obavlja zamjenu riječi s njenim sinonimom
- Omogućava pronalaženje dokumenata koji sadrže bilo traženu riječ bilo neki njen sinonim

1. C:\Program Files\PostgreSQL\17\share\tsearch_data> notepad synonym_sample.syn

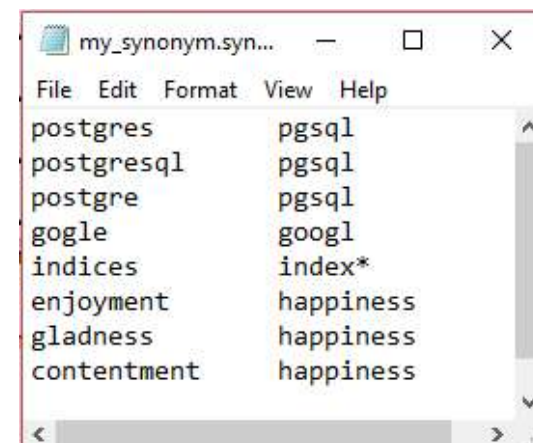


```
synonym_sample.syn...
File Edit Format View Help
postgres      pgsql
postgresql    pgsql
postgre       pgsql
gogle         googl
indices       index*
```

2. C:\Program Files\PostgreSQL\17\share\tsearch_data> copy synonym_sample.syn my_synonym.syn
C:\Program Files\PostgreSQL\17\share\tsearch_data> notepad my_synonym.syn

3.

```
CREATE TEXT SEARCH DICTIONARY my_xsyn
(TEMPLATE = synonym,
SYNONYMS = my_synonym);
```



```
my_synonym.syn...
File Edit Format View Help
postgres      pgsql
postgresql    pgsql
postgre       pgsql
gogle         googl
indices       index*
enjoyment     happiness
gladness      happiness
contentment   happiness
```

4.

```
SELECT ts_lexize
('my_xsyn', 'enjoyment');
```

ts_lexize

{happiness}

```
SELECT ts_lexize
('my_xsyn', 'gladness');
```

ts_lexize

{happiness}

PostgreSQL: Povezivanje parsera i rječnika

- Konfiguracijskim parametrima se parser povezuje sa skupom rječnika pomoću kojih se rezultat parsiranja dalje obrađuje.

```
CREATE TEXT SEARCH CONFIGURATION name (  
    PARSER = parser_name | COPY = source_config )
```

- Za svaki tip tokena definira se lista rječnika i redoslijed kojim se rječnici pozivaju pri normalizaciji tokena
- Rječnici se pozivaju redoslijedom kojim su navedeni u konfiguraciji
 - Ako rječnik prepozna token, rječnici nakon njega se ne konzultiraju
 - Ako rječnik ne prepozna token (vrati NULL), token se proslijeđuje sljedećem rječniku
- Obično se na početak liste stavlja najspecifičniji rječnik, potom općenitiji rječnici a na kraj liste najopćenitiji rječnik koji prepoznaje svaku riječ

PostgreSQL: Konfiguriranje opcija pretraživanja teksta

Inicijalno za tokene tipa *word* i *asciiword* se poziva samo *english_stem* rječnik:

```
SELECT *  
FROM ts_debug('english', 'Contentment is a state of mind.');
```

alias	description	token	dictionaries	dictionary	lexemes
asciiword	Word, all ASCII	Contentment	{english_stem}	english_stem	{content}
blank	Space symbols		{}	[null]	[null]
asciiword	Word, all ASCII	is	{english_stem}	english_stem	{}
blank	Space symbols		{}	[null]	[null]
asciiword	Word, all ASCII	a	{english_stem}	english_stem	{}
blank	Space symbols		{}	[null]	[null]
asciiword	Word, all ASCII	state	{english_stem}	english_stem	{state}
blank	Space symbols		{}	[null]	[null]
asciiword	Word, all ASCII	of		english_stem	{}
blank	Space symbols		{}	[null]	[null]
asciiword	Word, all ASCII	mind	{english_stem}	english_stem	{mind}
blank	Space symbols	.	{}	[null]	[null]

PostgreSQL: Konfiguriranje opcija pretraživanja teksta

Promijenimo postavke :

```
ALTER TEXT SEARCH CONFIGURATION english
ALTER MAPPING FOR word, asciiword WITH my_xsyn, english_stem;
```

```
SELECT *
FROM ts_debug('english', 'Contentment is a state of mind.');
```

alias	description	token	dictionaries	dictionary	lexemes
asciiword	Word, all ASCII	Contentment	{my_xsyn, english_stem}	my_xsyn	{happiness}
blank	Space symbols		{}	[null]	[null]
asciiword	Word, all ASCII	is	{my_xsyn, english_stem}	english_stem	{}
blank	Space symbols		{}	[null]	[null]
asciiword	Word, all ASCII	a	{my_xsyn, english_stem}	english_stem	{}
blank	Space symbols		{}	[null]	[null]
asciiword	Word, all ASCII	state	{my_xsyn, english_stem}	english_stem	{state}
blank	Space symbols		{}	[null]	[null]
asciiword	Word, all ASCII	of	{}	english_stem	{}
blank	Space symbols		{}	[null]	[null]
asciiword	Word, all ASCII	mind	{my_xsyn, english_stem}	english_stem	{mind}
blank	Space symbols	.	{}	[null]	[null]

PostgreSQL: TSVector

- TSVector

- podatkovni tip koji reprezentira dokument i optimiran je za FTS pa obuhvatnije pretražuje tekst od npr. LIKE, SIMILAR TO i ~
- sortirana lista leksema

primjeri preuzeti iz PostgreSQL dokumentacije

```
SELECT 'A fat cat sat on a mat and ate a fat rats'::TSVector;
```

TSVector
'A' 'a' 'on' 'and' 'ate' 'cat' 'fat' 'mat' 'rats' 'sat'

- Samo *castanje* na TSVector tip podatka ne provodi normalizaciju (uklanjanje stop riječi, svođenje na leksem,...), čak ne mijenja velika slova u mala
- Dokument možemo normalizirati pomoću funkcije *to_tsvector* :

```
SELECT to_tsvector ('english', 'A fat cat sat on a mat and ate a fat rats')
```

TSVector
'ate':9 'cat':3 'fat':2, 11 'mat':7 'rat':12 'sat':4

Zašto nema 'A',
'a', 'on', 'and'?

Što znače brojevi?
Čemu bi mogli služiti?

'rats'?

PostgreSQL: TSQuery

- podatkovni tip za predstavljanje upita s podrškom za Booleove operatore & (AND), | (OR) i ! (NOT)
- sastoji se od leksema povezanih Booleovim operatorima

```
SELECT 'The & fat & rats'::TSQuery;
```

tsquery
'The' & 'fat' & 'rats'

```
SELECT 'fat & (rats | cat)'::TSQuery;
```

tsquery
'fat' & ('rats' 'cat')

- funkcije *to_tsquery* i *plainto_tsquery* provode normalizaciju:

```
SELECT to_tsquery ('english', 'The & fat | rats')
```

to_tsquery
'fat' 'rat'

- *plainto_tsquery* lekseme uvijek povezuje s &:

```
SELECT plainto_tsquery ('english', 'The fat rats')
```

plainto_tsquery
'fat' & 'rat'

PostgreSQL: FTS operator @@

- operator za rad s TSVector i TSQuery
- rezultat je *true* ako TSVector (dokument) "odgovara" TSQuery (upitu)
- jednostavna podrška za FTS (bez rangiranja rezultata)

```
TSVector @@ TSQuery  
TSQuery  @@ TSVector
```

```
SELECT 'a fat cat sat on a mat and ate a fat rats'::TSVector @@  
       'cat & rat'::tsquery
```

?column?

false

```
SELECT 'a fat cat sat on a mat and ate a fat rat'::TSVector @@  
       'cat & rat'::tsquery
```

?column?

true

```
SELECT 'a fat cat sat on a mat and ate a fat rat'::TSVector @@  
       'cat & dog'::tsquery
```

?column?

false

PostgreSQL: ostali FTS operatori

@@	tsvector matches tsquery
	tsvector konkatencija
&, , !	tsquery AND, OR, NOT
<->	tsquery followed by tsquery
@>	tsquery contains
<@	tsquery is contained in

PostgreSQL: @@ brzina?

- Provođenje normalizacije dokumenata u velikom korpusu pri svakom obavljanju upita je sporo

ESPERIMENT

1. Tablica test100k sadrži 100 000 ntorki

```
CREATE TABLE test100K(  
    tekst    VARCHAR(2500)  
);
```

- Sadržaj preuzet s <http://www.anc.org/data/masc/downloads/data-download/>
- U tablici postoji 1 n-torka sa sadržajem
'Yeah well, I am lucky I will never lose'
- Testiram brzinu obavljanja upita:

```
SELECT * from test100k  
WHERE TO_tsVector('english', tekst)  
@@  
plainto_tsquery('english', 'Yeah well I am lucky I will never lose');
```

Trajanje: 3570 ms

PostgreSQL: @@ EKSPERIMENT

2. Tablica test1M sheme kao test100K sadrži 1 000 000 ntorki.
Sadržaj je udeseterostručeni sadržaj tablice test100K .
Trajanje ekvivalentnog upita: **27415ms (27 s)**.
3. Tablica test10M sheme kao test100K sadrži 10 000 000 ntorki
Sadržaj je ustostručeni sadržaj tablice test100K.
Trajanje ekvivalentnog upita: **277735 ms (4.63 min)!**

- Što napraviti?
- **Može li brže?**

tablica	Trajanje (ms)
test100K	3570 (3s)
test1M	27415 (27 s)
test10M	277735 (4.63 min)

PostgreSQL: @@ EKSPERIMENT

Poboljšanje, prva verzija:

- Dodati atribut tekstTSV tipa TSVector, koji će sadržavati aktualnu reprezentaciju teksta u normaliziranom obliku
- U upitima koristiti tekstTSV umjesto tekst

```
CREATE TABLE test100K
(
    tekst    VARCHAR(2500),
    tekstTSV TSVECTOR
);
```

```
CREATE TRIGGER test100K_InsUpd_Trigg
    BEFORE INSERT OR UPDATE ON test100K
    FOR EACH ROW
    EXECUTE PROCEDURE tsvector_update_trigger
        (tekstTSV, 'pg_catalog.english', tekst);
```

```
SELECT * from test100k
WHERE tekstTSV @@
    plainTO_tsquery('english', 'Yeah well I am lucky I will never lose');
```

Rezultati:

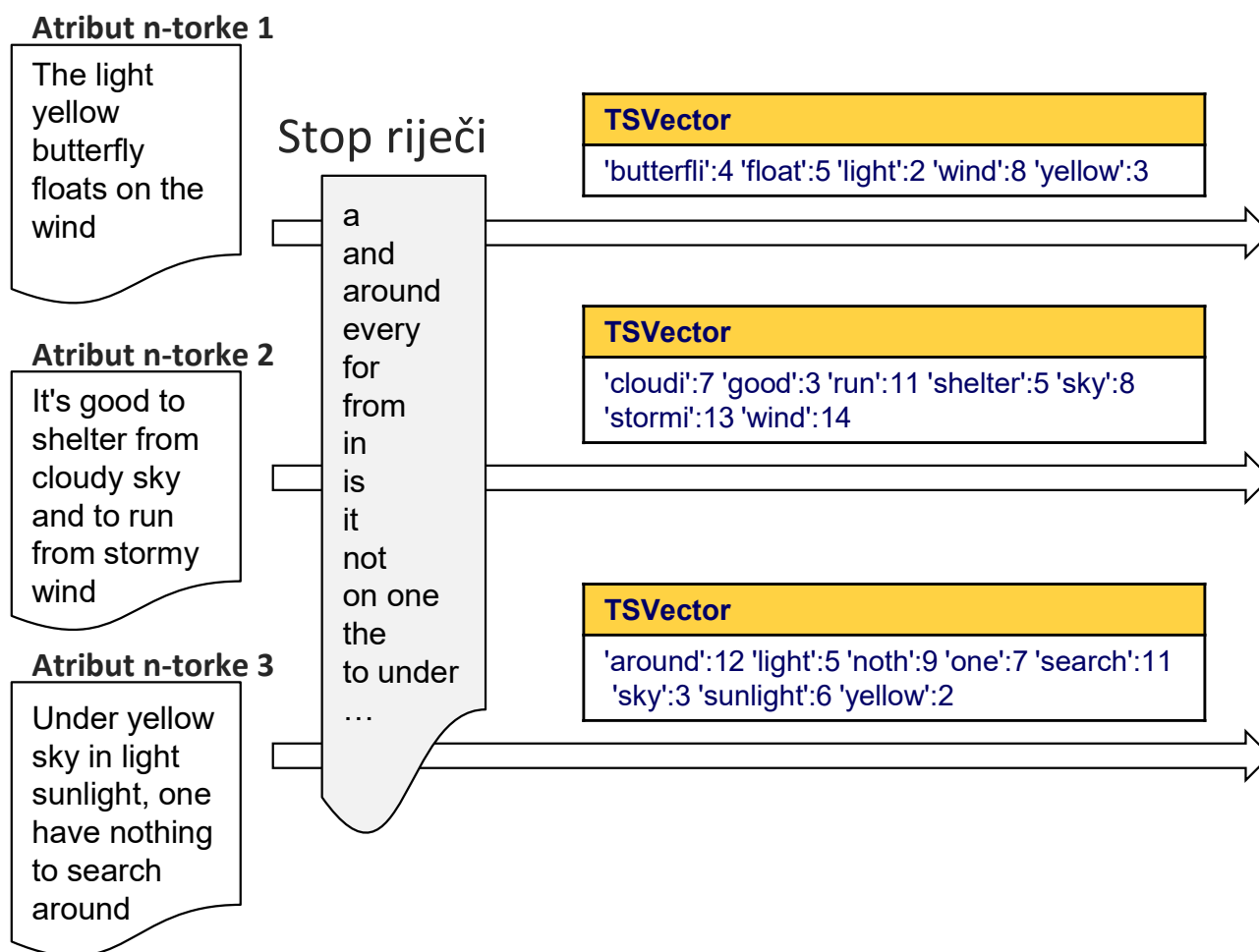
tablica	Trajanje (ms)	
	tekst	tekstTSV
test100K	3570 (3 s)	1000 (1s)
test1M	27415 (27 s)	9648 (9,6s)
test10M	277735 (4.63 min)	58920 (59s)

Može li brže?

Može! Kako?

Invertirani indeksi

- Ubrzavaju FTS
- Korisno je izraditi ga nad normaliziranim tekstom/dokumentom
- Naš tekst je sadržan u atributu n-torke



Invertirani indeks

ID	lexem	n-torka
1	butterfli	1
2	cloudi	2
3	float	1
4	good	2
5	light	1,3
6	noth	3
7	one	3
8	run	2
9	search	3
10	shelter	2
11	sky	2,3
12	stormi	
13	sunlight	3
14	wind	1, 2
15	yellow	1,3

PostgreSQL: Specijalni indeksi za FTS

- GIN indeks (Generalized Inverted Index)

- atribut mora biti tipa TSVector

```
CREATE INDEX idxName ON tableName USING gin(attrName)
```

- GIST indeks (Generalized Search Tree)

- atribut mora biti tipa TSVector ili TSQuery

```
CREATE INDEX idxName ON tableName USING gist(attrName)
```

- GIN indeksi

The GIN index type was designed to **deal with data types that are subdividable and you want to search for individual component values** (array elements, lexemes in a text document, etc)

- ✓ brža pretraga (oko 3x)
- ✗ dulje traje izgradnja indeksa (oko 3x)
- ✗ zauzimaju dva do tri puta više prostora za pohranu od GIST indeksa
- ✗ sporiji UPDATE

PostgreSQL: @@ Eksperiment

```
CREATE TABLE test10M
(
    tekst    VARCHAR(2500),
    tekstTSV TSVECTOR
);
```

```
CREATE TRIGGER test10M_InsUpd_Trigg
    BEFORE INSERT OR UPDATE ON test10M
    FOR EACH ROW
    EXECUTE PROCEDURE tsvector_update_trigger
        (tekstTSV, 'pg_catalog.english', tekst);
```

```
* CREATE INDEX ginTekstTsv10M ON test10M USING gin(tekstTSV);
** CREATE INDEX gistTekstTsv10M ON test10M USING gist(tekstTSV);
```

```
SELECT * from test100k
WHERE tekstTSV @@
      plainTO_tsquery('english', 'Yeah well I am lucky I will never lose');
```

Rezultati:

tablica	Trajanje (ms)				Veličina indeksa (MB)	
	tekst	tekstTSV	tekstTSV s GIN indeksom	tekstTSV s GIST indeksom	GIN*	GIST**
test100K	3570 (3 s)	1000 (1s)	31	31	71	10
test1M	27415 (27 s)	9648 (9,6s)	804	7071	640	100
test10M	277735 (4.63 min)	58920 (59s)	1942	100234	1102	1008

Rangiranje rezultata

- Prema relevantnosti dokumenta/znakovnog niza za postavljeni upit
 - Najbolji algoritam za rangiranje tek treba definirati
 - Osnovni elementi:
 - učestalost traženog uzorka/riječi u dokumentu
Koji dokument je relevantniji za traženi uzorak?
Dokument 1: duljina 1000 riječi traženi uzorak se pojavljuje 10 puta.
Dokument 2: duljina 1000 riječi traženi uzorak se pojavljuje 20 puta.
 - podudarnost traženog znakovnog niza s dokumentom (ili njegovim dijelom)
Koji dokument je relevantniji za traženi uzorak?
Dokument 1: traženi uzorak se pojavljuje na bliskim pozicijama u tekstu.
Dokument 2: riječi iz traženog uzorka su raštrkane po dokumentu.
 - u kojem dijelu dokumenta se traženi niz pojavljuje
Koji dokument je relevantniji za traženi uzorak?
Dokument 1: traženi uzorak se pojavljuje u tijelu dokumenta.
Dokument 2: traženi uzorak se pojavljuje u sažetku dokumenta.
- Korisno je znati na kojim pozicijama u tekstu se koja riječ pojavljuje

PostgreSQL: Rangiranje rezultata

Funkcija	Rezultat
<code>ts_rank([weights float4[],] vector tsvector, query tsquery [, normalization integer])</code>	Realni broj $\in [0, 1]$ relevantnost dokumenta za upit
<code>ts_rank_cd([weights float4[],] vector tsvector, query tsquery [, normalization integer])</code>	

- `ts_rank` rangira rezultate temeljem frekvencije leksema koji se podudaraju u upitu i dokumentu.
- `ts_rank_cd` računa rang prema radu Clarke, Cormack, and Tudhope's "Relevance Ranking for One to Three Term Queries". Slična je `ts_rank` osim što uzima u obzir udaljenost podudarnih leksema u uvjetu pretrage i pretraživanom tekstu
- Argument `weights` omogućuje pridjeljivanje različitih težina različitim dijelovima dokumenta
- Argument `normalization` omogućuje reguliranje utjecaja duljine dokumenta na rang.
- Za detalje možete pogledati PostgreSQL dokumentaciju
<https://www.postgresql.org/docs/current/textsearch-controls.html#TEXTSEARCH-RANKING>

PostgreSQL: Rangiranje rezultata

- Većina dokumenata ima sljedeće dijelove: naslov, ključne riječi, sažetak i tijelo
- Različitim dijelovima dokumenta ima smisla pri određivanju ranga dodijeliti različite težine – npr. najveću težini naslovu, a najmanju tijelu dokumenta
- U PostgreSQL-u se na to može utjecati pomoću polja *weights float4[]*
Pretpostavljene vrijednosti: {0.1, 0.2, 0.4, 1.0} = {D, C, B, A}

Funkcija	Rezultat
<code>setweight(vector tsvector, weight "char")</code>	tsvector - kopija ulaznog vektora u kojem je svaka pozicija označena danom "težinom"

document

title	keyword	abstract	body	allTSV
Ageing: Measuring our narrow strip of life	human; lifespan	In line with previous research, a demographic analysis corroborates the presence ...	The British author Annie Besant once wrote1: "out of the darkness of the womb, into the darkness of the grave, man passes across his narrow strip of life."...	'across':43 'age':1A 'analysi':17C 'anni':24 'author':23 'besant':25 'british':22 'corrobor':18C 'dark':31,37 (...)
...	...	...	...	...

```
UPDATE document
SET allTSV = setweight(to_tsvector('english', title), 'A') ||
             setweight(to_tsvector('english', keyword), 'B') ||
             setweight(to_tsvector('english', abstract), 'C') ||
             setweight(to_tsvector('english', body), 'D');
```

PostgreSQL: ts_rank

```
SELECT ts_rank(to_tsvector('The scientists at Westworld noticed aberrant  
behavior of the android inhabitants'),  
to_tsquery('inhabitants'));
```

ts_rank

0.0607927

```
SELECT ts_rank(to_tsvector('The scientists at Westworld noticed aberrant  
behavior of the android inhabitants'),  
to_tsquery('behavior & inhabitants'));
```

ts_rank

0.095243

```
SELECT ts_rank(to_tsvector('The scientists at Westworld noticed aberrant  
behavior of the android inhabitants'),  
to_tsquery('scientists & aberrant & behavior & inhabitants'));
```

ts_rank

0.422803

- D je pretpostavljena "težina" leksema u ts_rank. Ako pridijelim veću težinu leksemima, i rang se mijenja

```
SELECT ts_rank(setweight(to_tsvector('The scientists at Westworld noticed aberrant  
behavior of the android inhabitants'), 'C'),  
to_tsquery('scientists & aberrant & behavior & inhabitants'));
```

ts_rank

0.685564

PostgreSQL: utjecaj duljine dokumenta

D1	<pre>SELECT ts_rank(to_tsvector('The scientists at Westworld noticed aberrant behavior of the android inhabitants'), to_tsquery('behavior & inhabitants'));</pre>	<table><tr><th>ts_rank</th></tr><tr><td>0.095243</td></tr></table>	ts_rank	0.095243
ts_rank				
0.095243				
D2	<pre>SELECT ts_rank(to_tsvector('The scientists at Westworld noticed aberrant behavior of the android inhabitants. The scientists at Westworld noticed aberrant behavior of the android inhabitants'), to_tsquery('behavior & inhabitants'));</pre>	<table><tr><th>ts_rank</th></tr><tr><td>0.249922</td></tr></table>	ts_rank	0.249922
ts_rank				
0.249922				
D3	<pre>SELECT ts_rank(to_tsvector('The scientists at Westworld noticed aberrant behavior of the android inhabitants. The scientists at Westworld noticed aberrant behavior of the android inhabitants. The scientists at Westworld noticed aberrant of the android '), to_tsquery('behavior & inhabitants'));</pre>	<table><tr><th>ts_rank</th></tr><tr><td>0.249922</td></tr></table>	ts_rank	0.249922
ts_rank				
0.249922				

Frekvencije: D1 'behavior'- 1 'inhabit' – 1

D2 'behavior'- 2 'inhabit' – 2

D3 'behavior'- 2 'inhabit' – 2

Dokument D3 ima jednak rang kao D2, ali je za (malo manje od) trećinu dulji.

Treba li on doista imati jednak rang kao D2?

Treba li D3 imati veći rang od D1?

PostgreSQL: utjecaj duljine dokumenta

Funkcija

`ts_rank([ weights float4[], ] vector tsvector, query tsquery [, normalization integer])`

normalization $\in \{0, 1, 2, 4, 8, 16, 32\}$ omogućuje reguliranje utjecaja duljine dokumenta na rang.

0 (default) ne uzima u obzir duljinu dokumenta

1 dijeli rang s $1 + \log$ od duljine dokumenta

2 dijeli rang s duljinom dokumenta

...

D1	<pre>SELECT ts_rank(to_tsvector('The scientists at Westworld noticed aberrant behavior of the android inhabitants'), to_tsquery('behavior & inhabitants'), 2);</pre>	<table><tr><th>ts_rank</th></tr><tr><td>0.00865845</td></tr></table>	ts_rank	0.00865845
ts_rank				
0.00865845				
D2	<pre>SELECT ts_rank((to_tsvector('The scientists at Westworld noticed aberrant behavior of the android inhabitants. The scientists at Westworld noticed aberrant behavior of the android inhabitants'), to_tsquery('behavior & inhabitants'), 2);</pre>	<table><tr><th>ts_rank</th></tr><tr><td>0.0113601</td></tr></table>	ts_rank	0.0113601
ts_rank				
0.0113601				
D3	<pre>SELECT ts_rank(to_tsvector('The scientists at Westworld noticed aberrant behavior of the android inhabitants. The scientists at Westworld noticed aberrant behavior of the android inhabitants. The scientists at Westworld noticed aberrant of the android '), to_tsquery('behavior & inhabitants'), 2);</pre>	<table><tr><th>ts_rank</th></tr><tr><td>0.00806199</td></tr></table>	ts_rank	0.00806199
ts_rank				
0.00806199				

Jesu li ovi rangovi prikladniji?

PostgreSQL: FTS operator @@ - primjer

movies

movieid	title	To_tsvector(title)
67	Dirty Dancing	"danc':2 'dirt':1"
6569	Dances with Wolves	"danc':1 'wolv':3"
78	Shall We Dance?	"danc':3 'shall':1"
368	The Dancing Masters	"danc':2 'master':3"
90634	The Man Who Loved Cat Dancing	"cat':5 'danc':6 'love':4 'man':2"

```
SELECT title FROM movies
WHERE title::TSVector @@ 'Dancing'::TSQuery
```

title
Dirty Dancing
The Dancing Masters
The Man Who Loved Cat Dancing

```
SELECT title FROM movies
WHERE to_TSVector(title) @@ to_TSQuery('Dancing')
```

title
Dirty Dancing
Dances with Wolves
Shall We Dance?
The Dancing Masters
The Man Who Loved Cat Dancing

```
SELECT title FROM movies
WHERE to_TSVector(title) @@ to_TSQuery('Dencing')
```

title

Približno pretraživanje teksta (Fuzzy Text Search)

- Tehnika pronalaženja dokumenta/niza znakova koji se približno podudara s traženim uzorkom
- Kvaliteta podudaranja se mjeri ovisno o vrsti primijenjenog algoritma npr.
 - brojem operacija koje je potrebno obaviti nad znakovnim nizom da bi se u potpunosti podudario s traženim uzorkom
 - brojem podudanih podnizova i sl.
- različite vrste algoritama:
 - algoritmi temeljeni na udaljenosti znakovnih nizova
 - Hamming, **Levenshtein**
 - Q-Gram algoritmi
 - slični znakovni nizovi imaju više zajedničkih Q-Gram-ova (podskupovi znakovnog niza duljine Q)
 - Fonetski algoritmi
 - Soundex, Metaphone algoritam
 - traže riječi koje se slično izgovaraju

Algoritmi temeljeni na udaljenosti znakovnih nizova

- Udaljenost uređivanja („edit distance“) znakovnih nizova s_1 i s_2 je minimalan broj operacija potrebnih da se jedan niz transformira u drugi. Moguće operacije su: izmjena, umetanje, brisanje znaka.
- Autor ideje je Vladimir Levenshtein 1965. - Levenshteinova udaljenost
- Rješenje ne mora biti jednoznačno
- Problem se svodi na pronalaženje slijeda navedenih operacija kojim će se jedan niz transformirati u drugi uz minimalan trošak (operacije ne moraju jednako „koštati“)

Primjer: odrediti udaljenost između SREĆA i SRETNA (tako da je odredišni niz SREĆA)

Pokušaj 1

1. Zamijeniti T sa Ć

2. Zamijeniti N s A

3. Obrisati A

Udaljenost = 3

S	R	E	Ć	A	
S	R	E	T	N	A

S	R	E	Ć	A	
S	R	E	Ć	N	A

S	R	E	Ć	A	
S	R	E	Ć	A	A

S	R	E	Ć	A	
S	R	E	Ć	A	

Pokušaj 2

1. Zamijeniti T sa Ć

2. Obrisati N

Udaljenost = 2

S	R	E	Ć	A	
S	R	E	T	N	A

S	R	E	Ć	A	
S	R	E	Ć	N	A

S	R	E	Ć	A	
S	R	E	Ć	A	

Je li broj operacija manji ako mijenjam SREĆA u SRETNA?

Levenshteinova udaljenost

Koliko košta da nakon obavljenih operacija s1 i s2 budu = s2 tj. SRETNA?

		S	R	E	T	N	A
	0	1	2	3	4	5	6
S	1						
R	2						
E	3						
Ć	4						
A	5						

S2 - target

S1 - source

```

m = len(s1);
n = len(s2);
Inicijaliziraj prvi redak (0...n);
Inicijaliziraj prvi stupac (0...m);
for (i=1 to m)
    for (j=1 to n)
        ako je s1[i] == s2[j]
            trošak=0;
        inače
            trošak = 1;
        d[i,j] = min(d[i-1,j] + 1;
                    d[i,j-1] + 1;
                    d[i-1,j-1] + trošak)
Udaljenost = d[m,n];
    
```

		S
	0	1
S	1	0

1+1
0+0

		S	R
	0	1	2
S	1	0	1

0+1
1+1

Kretanje

udesno : umetanje znaka (u s1)

dolje : brisanje (iz s1)

dijagonalno: slaganje ili neslaganje uz izmjenu

Levenshteinova udaljenost

Koliko košta da nakon obavljenih operacija s1 i s2 budu = s2 tj. SRETNA?

		S	R	E	T	N	A	s2
	0	1	2	3	4	5	6	
S	1	0	1	2	3	4	5	
R	2	1	0	1	2	3	4	
E	3	2	1	0	1	2	3	
Ć	4	3	2	1	1	2	3	
A	5	4	3	2	2	2	2	

s1

Kretanje

udesno : umetanje znaka (u s1) $(d[i-1,j]+1)$
 dolje : brisanje (iz s1) $(d[i,j-1]+1)$
 dijagonalno: slaganje ili neslaganje uz izmjenu $(d[i-1,j-1]+trošak)$

- ① s1 izmjena: Ć postaje T (s1= SRETA)
- ② s1 umetanje: N iza T (s1 = SRETNA)
- ② s1 bez operacije

Levenshteinova udaljenost

Koliko košta da nakon obavljenih operacija s1 i s2 budu = SREĆA?

		S	R	E	Ć	A	s2
	0	1	2	3	4	5	
S	1	0	1	2	3	4	
R	2	1	0	1	2	3	
E	3	2	1	0	1	2	
T	4	3	2	1	1	2	
N	5	4	3	2	2	2	
A	6	5	4	3	3	2	
s1							

Rezultat mora biti jednak kao u prethodnom postupku

Kretanje

udesno : umetanje znaka (u s1)

dolje : brisanje (iz s1)

dijagonalno: slaganje ili neslaganje uz izmjenu

1

s1 izmjena:

T postaje Ć (s1=SREĆNA)

2

s1 brisanje:

N (s1 =SREĆA)

3

s1 bez operacije

Linkovi za dodatna pojašnjenja:

<http://www-igm.univ-mlv.fr/~lecroq/seqcomp/node1.html>

<http://people.cs.pitt.edu/~kirk/cs1501/Pruhs/Fall2006/Assignments/editdistance/Levenshtein%20Distance.htm>

<http://www.csse.monash.edu.au/~lloyd/tildeAlgDS/Dynamic/Edit/>

<http://www-igm.univ-mlv.fr/~lecroq/seqcomp/node2.html>

PostgreSQL: Levenshtein

Funkcija	Rezultat
levenshtein(text source , text target , int ins_cost, int del_cost, int sub_cost)	Cijeli broj - udaljenost između source i target
levenshtein(text source , text target)	

```
SELECT levenshtein ('The Dancing Masters', 'The Dancing Masters') lev1
      , levenshtein ('The Dancing Masterss', 'The Dencing Master') lev2v1
      , levenshtein ('The Dencing Master', 'The Dancing Masterss') lev2v2
      , levenshtein ('The Dancing Masterss', 'Dancing Master') lev3v1
      , levenshtein ('Dancing Master', 'The Dancing Masterss') lev3v2
```

lev1	lev2v1	lev2v2	lev3v1	lev3v2
0	2	2	5	5

```
SELECT levenshtein ('The Dancing Masters', 'The Dancing Master', 3, 2, 1) lev1
      , levenshtein ('The Dancing Master', 'The Dancing Masterss', 3, 2, 1) lev2
      , levenshtein ('The Dancing Mastery', 'The Dancing Masterss', 3, 2, 1) lev3
```

ins=3, del=2, upd=1

lev1	lev2	lev3
2	3	1

Link za dodatna pojašnjenja:

<https://www.postgresql.org/docs/current/fuzzystmatch.html>

Q-Gram algoritmi

- Tekst/dokument je (multi) skup n-grama
- n-gram može biti
 - riječ
 - znakovni niz, podskup teksta, duljine Q znakova
- Teza: ako je riječ A slična riječi B, one vjerojatno sadrže barem jedan podudaran (zajednički) podniz duljine Q

Primjer: $q = 2$ (bigrami)

happiness:	ha	ap	pp	pi	in	ne	es	ss
happily:	ha	ap	pp	pi	il	ly		

- 4 podudarna od ukupno 10 bigrama (bez ponavljanja podudarnih)
- mjera sličnosti bi se mogla odrediti kao kvocijent: $4/10 = 0,4$

Q-Gram algoritmi

- Q-gram algoritam ne radi uvijek dobro – za kraće riječi i veći q

Primjer: $q = 3$ (trigrami)

votka : vot otk tka

vodka : vod odk dka

- 0 podudarnih od ukupno 6 trigrama $\Rightarrow$ sličnost iznosi 0

- Prednost ove kategorije algoritama: primjenjivi bez obzira na jezik i domenu

PostgreSQL: Q-Gram

- $Q = 3$ - trigram
- PostgreSQL dodaje 2 praznine na početak i 1 prazninu na kraj svakog niza

Primjer:

happiness	__h	_ ha	hap	app	ppi	pin	ine	nes	ess	ss_
happily	__h	_ ha	hap	app	ppi	pil	ily	ly_		

- 5 podudarnih od ukupno 13 trigrama (bez ponavljanja podudarnih)
- sličnost: $5/13 = 0,384615$

PostgreSQL: Trigram

Funkcija	Rezultat
<code>similarity(text, text)</code>	Realni broj $\in [0, 1]$ – mjera podudarnosti dva argumenta. 0 za potpuno različite, 1 za identične argumente.
<code>show_trgm(text)</code>	Polje trigrama ulaznog znakovnog niza.
<code>show_limit()</code>	Trenutni prag za podudarnost kojeg koristi operator %. Prag predstavlja minimalnu sličnost koja mora postojati između dva znakovna niza da bi se smatrali "sličnima".
<code>set_limit(real)</code>	Postavlja vrijednost praga podudarnosti kojeg koristi operator %. Prag mora biti između 0 i 1. Inicijalna vrijednost je 0.3.
Operatori koji koriste funkciju similarity	
<code>text % text</code>	<i>True</i> ako je podudarnost među argumentima veća od trenutnog praga podudarnosti .

Link za dodatna pojašnjenja:

<https://www.postgresql.org/docs/current/pgtrgm.html>

PostgreSQL: Trigram - primjeri

```
SELECT show_trgm('happiness')
```

Show_trgm

"{" h"," ha",app,ess,hap,ine,nes,pin,ppi,"ss {"

```
SELECT show_trgm('happily')
```

Show_trgm

"{" h"," ha",app,hap,ily,"ly ",pil,ppi}"

```
SELECT similarity('happiness', 'happily')
```

similarity

0.384615

sličnost: $5/13 = 0,384615$

```
SELECT title, similarity (title, 'The Dencing Master')
FROM movies
WHERE title % 'The Dencing Master'
-- WHERE similarity (title, 'The Dencing Master') > 0.5
```

title	similarity
The Dancing Masters	0.625

Restriktivniji uvjet
od inicijalnog praga

PostgreSQL: Trigram

- može biti jako sporo za puno velikih dokumenata
- mogućnost kreiranja specijalnog (invertiranog) indeksa
- Invertirani indeks brzo "uparuje" vektor trigrama dokumenta s

```
CREATE INDEX title_trigram_idx ON movies USING gist(title gist_trgm_ops);
```

- *gist_trgm_ops* je opcija koja osigurava kreiranje indeksa za trigrame

PostgreSQL: Soundex i Metaphone

- Aktualno u jezicima u kojima se izgovor riječi razlikuje od zapisa (ne pretjerano korisno za hrvatski)
- Ideja je dovesti u vezu riječi koje se jednako ili slično izgovaraju ali drugačije zapisuju
- Algoritam riječ predstavlja znakovnim nizom koji prezentira izgovor (zvučanje) riječi
- Soundex je u početku bio ograničen na vlastita imena u engleskom jeziku

```
CREATE EXTENSION fuzzystmatch;  
SELECT soundex('Anne') s1, soundex('Ann') s2, soundex('Anna') s3
```

s1	s2	s3
A500	A500	A500

- Metaphone koristi ideju algoritma Soundex ali uvodi poboljšanja (dijakritici,...)

```
metaphone(text source, int maxLength) returns text
```

```
SELECT metaphone ('Dancing',7) m1, metaphone('Dencing',7) m2
```

m1	m2
TNSNK	TNSNK

```
SELECT metaphone ('Bruce Wilis',7) m1  
      , metaphone ('Broos Wils',6) m2
```

m1	m2
BRSWLS	BRSWLS

PostgreSQL: Približno pretraživanje teksta - sažetak

- Levenshtein
 - prikladan za pronalaženje znakovnih nizova koji sadrže neznatne razlike (manje pravopisne pogreške) ali moraju biti jako slični
- Trigram
 - prikladan i za veća odstupanja (pravopisne pogreške)
- Soundex, Metaphone
 - prikladan za pronalaženje riječi (znakovnih nizova) koje se jednako ili slično izgovaraju